

این گزارش، مستندات کامل و گام‌به‌گام طراحی، پیاده‌سازی و راستی‌آزمایی یک «پردازنده ساده تک‌پشته‌ای (Stack-Based CPU) ۸ بیتی» روی FPGA است.

در ادامه، کل پروژه از صفر تا صد، با زبانی ساده، دسته‌بندی‌شده و دقیق تشریح شده است:

۱. هدف و صورت کلی مسئله

هدف این آزمایش، طراحی یک ریزپردازنده ۸ بیتی با معماری مبتنی بر پشته (Stack) است. بر خلاف پردازنده‌های رجیستری (مثل MIPS یا ARM)، در معماری پشته‌ای بیشتر عملیات محاسباتی بدون نیاز به نام‌بردن از رجیسترها و مستقیماً روی داده‌های بالای پشته انجام می‌شوند.

وظیفه نهایی پردازنده: پردازنده باید ورودی ۸ بیتی X را از کلیدها دریافت کرده و مقدار فرمول زیر را محاسبه و روی نمایشگر سون‌سگمنت و LEDها نمایش دهد:

$$Y = ((X + 23) \times 2) - 12$$

۲. معماری سخت‌افزار (Datapath & Memory)

۱. عرض مسیر داده (Data Width): تمام مسیرها، ثبات‌ها و عملیات حسابی ۸ بیتی هستند.

۲. پشته سخت‌افزاری (Hardware Stack):

- شامل ۸ خانه ۸ بیتی داخلی است.

- یک اشاره‌گر پشته (SP) به عرض ۴ بیت دارد که تعداد عناصر موجود در پشته (0 تا 8) را نگه می‌دارد.

- مکانیزم حفاظتی در برابر سرریز (Overflow: $SP > 8$) و کم‌ریز (Underflow):

SP < 0) تعبیه شده که در صورت بروز خطا، سیگنال stack_error فعال می‌شود.

3. حافظه اصلی (Memory Map):

- یک فضای یکپارچه ۲۵۶ بایتی (آدرس‌های h00 تا FFh) برای کد برنامه و داده‌ها وجود دارد.

- از منطق ورودی/خروجی نگاشت‌شده در حافظه (MMIO) استفاده شده است:

- آدرس F8h: درگاه خواندن ورودی X (فقط خواندنی).

- آدرس F9h: درگاه نوشتن خروجی Y (قابل خواندن و نوشتن).

- آدرس‌های FAh تا FFh: برای توسعه رزرو شده‌اند.

- آدرس‌های h00 تا F7h: حافظه عادی داده و کد.

4. ثبات‌های پردازنده:

- PC (شمارنده برنامه - ۸ بیت): آدرس دستور بعدی.

- IR (ثبات دستور - ۸ بیت): نگهداری کد دستور جاری.

- Operand (ثبات عملوند - ۸ بیت): نگهداری مقدار ثابت یا آدرس در دستورهای دو بایتی.

- پرچم‌ها (Flags): دو پرچم Z (صفر) و S (علامت/منفی) که صرفاً با دستورهای

ADD و SUB تغییر می‌کنند.

۳. مجموعه دستورات (ISA)

Opcode هر دستور در ۴ بیت بالایی بایت اول قرار دارد. دستورها به دو دسته یک‌بایتی و دو بایتی تقسیم می‌شوند:

آپکد (Hex)	نام دستور	طول	توضیح عملکرد
-----	-----	----	-----
-----	-----	----	-----

`0` | ** | ** | PUSHC C** ۲ بایت | مقدار ثابت ۸ بیتی \$C\$ را در بالای پشته قرار می‌دهد.
 |
 `1` | ** | ** | PUSH M** ۲ بایت | مقدار خانه حافظه (یا ورودی MMIO) در آدرس \$M\$ را خوانده و Push می‌کند.
 |
 `2` | ** | ** | POP M** ۲ بایت | مقدار بالای پشته را برداشته و در آدرس \$M\$ (یا خروجی MMIO) می‌نویسد.
 |
 `3` | ** | ** | JUMP** ۱ بایت | آدرس پرش را از بالای پشته POP کرده و در `PC` می‌ریزد.
 |
 `4` | ** | ** | JZ** ۱ بایت | آدرس را POP می‌کند؛ اگر Z=1\$ باشد پرش می‌کند (برای تعادل پشته همیشه POP انجام می‌شود).
 |
 `5` | ** | ** | JS** ۱ بایت | آدرس را POP می‌کند؛ اگر S=1\$ باشد پرش می‌کند (همیشه POP انجام می‌شود).
 |
 `6` | ** | ** | ADD** ۱ بایت | دو عنصر بالای پشته را POP کرده، جمع می‌زند و حاصل را Push می‌کند (\$Z\$ و \$S\$ تنظیم می‌شوند).
 |
 `7` | ** | ** | SUB** ۱ بایت | عنصر بالای پشته را از عنصر زیرین کم می‌کند (\$TOS_{-1}\$ - \$TOS\$) و حاصل را Push می‌کند (\$Z\$ و \$S\$ تنظیم می‌شوند). |

۴. واحد کنترل و ماشین حالت (FSM)

کنترلر پردازنده دارای ۴ وضعیت (State) است:

1. ST_FETCH: خواندن کد دستور از memory[PC] درون IR و افزایش PC.
2. ST_DECODE:
 - اگر دستور ۱ بایتی باشد (ADD, SUB, JUMP, JZ, JS)، فوراً اجرا شده و وضعیت به ST_FETCH بازمی‌گردد.
 - اگر دستور ۲ بایتی باشد (PUSHC, PUSH, POP)، وضعیت به ST_OPERAND می‌رود.
3. ST_OPERAND: خواندن بایت دوم (عملوند) از memory[PC] درون ثبات Operand و افزایش PC.
4. ST_EXEC_OPERAND: اجرای عملیات مربوط به دستور ۲ بایتی و بازگشت به ST_FETCH.

۵. برنامه نمونه (محاسبه فرمول)

چون پردازنده دستور ضرب (MUL) ندارد، ضرب در ۲ از طریق «ذخیره موقت + دو بار Push + یک بار ADD» انجام شده است:

; آدرس | کد هگز | کد اسمبلی | توضیح
01-00 | PUSH F8h 10 | F8 | خواندن X از ورودی و قرار دادن در پشته
03-02 | PUSHC 23 | 17 00 | عدد 23 را Push کن
04 | ADD | 60 | محاسبه (X + 23)
06-05 | POP 20h | 20 20 | نتیجه در آدرس موقت h20 ذخیره شود
08-07 | PUSH 20h | 20 10 | نسخه اول (X + 23)
09-20h | PUSH 20h | A | 10 20 | نسخه دوم (X + 23)
ADD0 | B | 60 | جمع دو نسخه = ضرب در 2
120 | PUSHC 120 | C-0D | 00 0C | عدد 12 را Push کن
SUB0 | E | 70 | تفریق 12 از مقدار قبلی
F9h0 | POP F9h | F-10 | 20 F9 | ارسال نتیجه نهایی Y به درگاه خروجی
12-11 | PUSHC 0 | 00 00 | آدرس شروع (0) برای حلقه
13 | JUMP | 30 | پرش به آدرس 0 و تکرار همیشگی

۶. ساختار ماژول‌های RTL (کد SystemVerilog)

پروژه به ۳ ماژول اصلی تقسیم شده است:

1. stack_cpu.v (هسته پردازنده):

- شامل FSM، آرایه پشته، آرایه ۲۵۶ بایتی حافظه (با مقادیر اولیه برنامه در

بلاک initial)، ثبات‌ها و سیگنال‌های کنترلی.

2. seven_segment.v (راه‌انداز سون‌سگمنت):

- ورودی ۸ بیتی را به‌صورت عدد علامت‌دار مکمل دو در نظر می‌گیرد.
- مقدار قدرمطلق را به ارقام یکان، دهگان و صدگان تجزیه کرده و با تقسیم زمانی (Multiplexing با کانتر ۱۶ بیتی) روی ۴ رقم نمایش می‌دهد.
- رقم چهارم برای نمایش علامت منفی (-) در صورت منفی بودن عدد استفاده می‌شود.

3. cpu_system.v (ماژول سطح بالا - Top Level):

- هسته پردازنده را به سون‌سگمنت و پین‌های ورودی/خروجی وصل می‌کند.
- تشخیص خطای بازه (Range Check): مقدار ورودی به ۱۰ بیت گسترش علامت یافته و فرمول به‌صورت ریاضی محاسبه می‌شود؛ اگر مقدار حاصله خارج از بازه قابل نمایش [127- , 127+] باشد یا خطای پشته رخ دهد، چراغ error_led روشن می‌شود.

۷. شبیه‌سازی و اعتبارسنجی (Verification)

سه تست‌بنچ خودآزما (Self-Checking) برای این پروژه نوشته شده است:

1. tb_stack_cpu: تکتک دستورها، صحت پرچم‌های Z و S، ترتیب تفریق در SUB، پرش‌های

شرطی و مسیر MMIO را بررسی می‌کند.

2. tb_cpu_system_exhaustive: برنامه را برای تمام ۲۵۶ مقدار ممکن ورودی 0 تا X

255 اجرا کرده و نتیجه سخت‌افزار را با مدل محاسباتی مقایسه می‌کند (۱۰۰٪)

تطابق).

3. tb_display_range: فعال شدن چراغ خطای بازه (error_led) را برای تمامی حالات

خارج از بازه علامت‌دار بررسی می‌کند.

۸. سنتز و منابع سخت‌افزاری (FPGA Implementation)

- مدار در نرم‌افزار Quartus II برای خانواده Cyclone IV سنتز شده است.
- تعداد کل پایه‌های I/O اسکالر در مازول تاپ برابر ۲۳ پین است (شامل کلاک، ریست، ۸ کلید ورودی، ۷ سگمنت، ۴ فعال‌ساز آند و ۲ عدد LED).
- مصرف منابع، دیاگرام RTL Viewer و شبیه‌سازی Post-Mapping همگی در گزارش درج شده و نشان‌دهنده سنتزپذیر بودن کامل کد هستند.